下载 SDL 并引入

下载 SDL

SDL 官网地址：http://www.libsdl.org/
我们现在都是在 Windows 系统下使用，因此直接下载编译好的版本就行了。
基本上的库 Windows 的版本都有提供已经编译好的版本，SDL 也是如此：



下载 Runtime 库 SDL2-2.0.10-win32-x86.zip(运行时库 dll) 以及 Development 库 SDL2-devel-2.0.10-VC.zip(编译时 lib)

引用 SDL

尽量选用现成的东西来实现我们的功能，而不是啥都自己动手，比如这里我们选择 SDL 来播放音频。

这里我们要实现的是使用 FFMPEG+SDL 来播放音频文件，FFMPEG 用来解码音频, SDL 用来播放声音.

将 ffmpeg 和 SDL2 拷贝到工程目录下，在工程的 pro 文件中加入：

```
INCLUDEPATH += $$PWD/ffmpeg-4.2.2/include\
               $$PWD/SDL2-2.0.10/include

LIBS += $$PWD/ffmpeg-4.2.2/lib/avcodec.lib\
        $$PWD/ffmpeg-4.2.2/lib/avdevice.lib\
```

```
        $$PWD/ffmpeg-4.2.2/lib/avfilter.lib\
        $$PWD/ffmpeg-4.2.2/lib/avformat.lib\
        $$PWD/ffmpeg-4.2.2/lib/avutil.lib\
        $$PWD/ffmpeg-4.2.2/lib/postproc.lib\
        $$PWD/ffmpeg-4.2.2/lib/swresample.lib\
        $$PWD/ffmpeg-4.2.2/lib/swscale.lib\
        $$PWD/SDL2-2.0.10/lib/x86/SDL2.lib
```

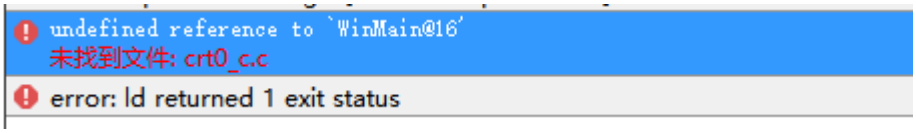
这里是同时引用了 FFMPEG 和 SDL 的工程文件。

包含头文件

```
extern "C"
{
    #include "libavcodec/avcodec.h"
    #include "libavformat/avformat.h"
    #include "libavutil/pixfmt.h"
    #include "libswscale/swscale.h"
    #include "libavformat/avformat.h"
    #include "libavdevice/avdevice.h"
    #include <SDL.h>
}
#define SDL_AUDIO_BUFFER_SIZE 1024
#define AVCODEC_MAX_AUDIO_FRAME_SIZE 192000 // 1 second of 48khz 32bit audio
//192000 的由来: 播放的音乐是 48khz 32 位的音频 这里主要计算采样率 就是 1s 采样多少字节
//32bit-> 4 字节 48khz 是 1s 采样 48000 次 每次 4 字节 ， 也就是 1s 采样 48000*4 = 192000 字
节
```

注意事项

加入 SDL 的头文件之后，编译的时候会提示 main 函数没有定义



原因是 SDL_main.h 中有如下一段话:

```
#if defined(SDL_MAIN_NEEDED) || defined(SDL_MAIN_AVAILABLE)
#define main    SDL_main
#endif
```

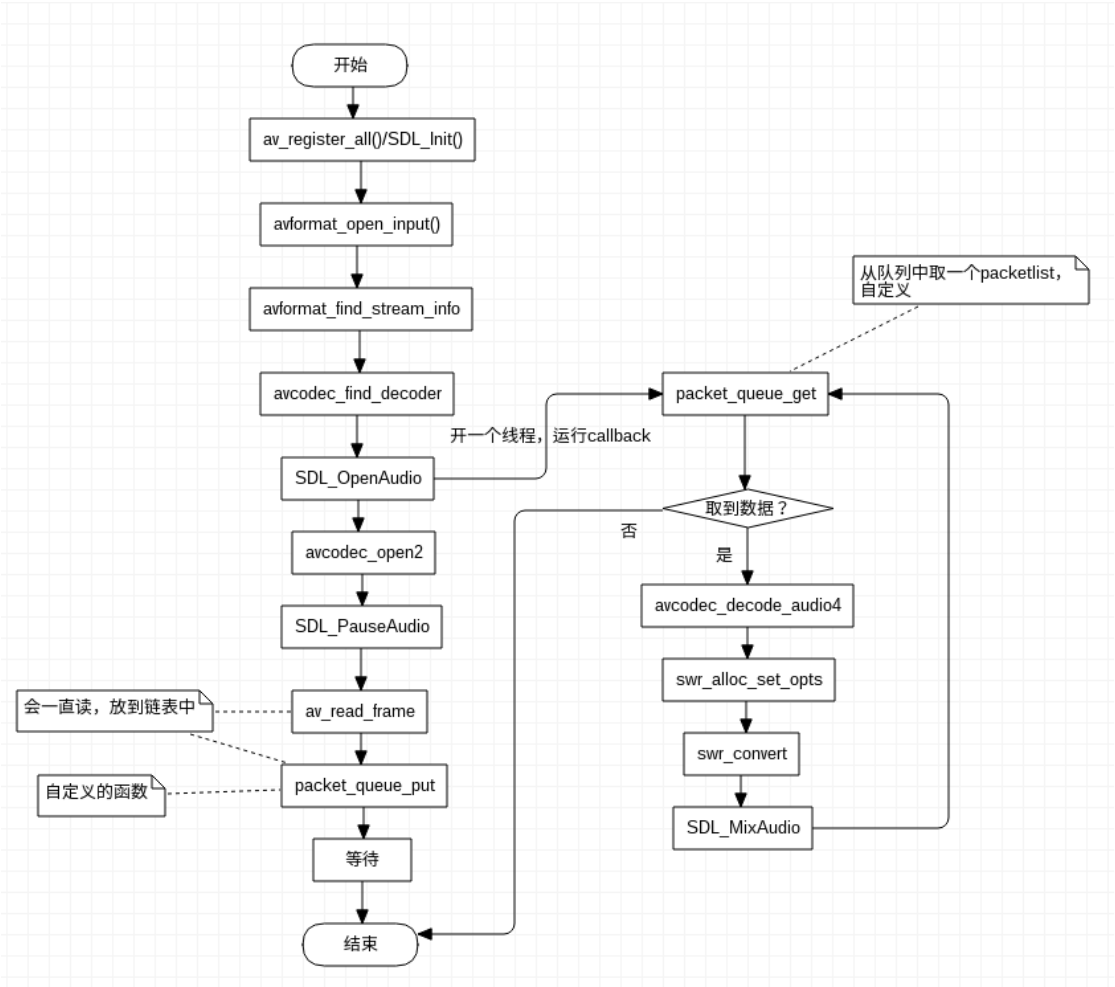
可以看出这里已经定义了一个 main 了，因此我们写的程序中的 main 便不能生效了，解决方法：
在我们自己写的 main 函数的前面加上

```
#undef main
int main(int argc, char *argv[])
```

```
{
    cout << "Hello World!" << endl;
    return 0;
}
```

ffmpeg+SDL 播放音乐流程

FFMPEG 读取音频文件和解码音频的代码，基本上和前面视频的操作类似。
这里需要注意的一点是：
SDL 播放音频是通过回调函数的方式播放，且这个回调函数是在新的线程中运行，此回调函数固定时间激发一次，这个时间和要播放的音频频率有关系。
因此我们用 FFMPEG 读到一帧音频后，不是急着解码，而是将数据存入一个队列，等 SDL 回调函数激发的时候，从这个队列中取出数据，然后解码播放。



主要思想是：
主线程只负责读 AVPacket 存到队列。→ av_read_frame()
其他所有的解码，输出工作都由回调函数 callback 完成。
callback 中从队列中取 AVPacketList，再把 AVPacketList 中的 AVPacket 拿出来解码。
解码后放到缓冲区，然后输出。
一次调用 callback，就输出长度为 len 的数据（callback 第三个参数，一般为 2048），不多不少。

当缓冲区没有数据的时候，就去解码，放到缓冲区，再输出。
当解码的数据多于 len 的时候，就只处理 len 个，其他的留给后续的 callback。
callback 函数会一次次调用，直到所有处理完。

```
//0.申请变量
//1.ffmpeg 初始化
//2.SDL 初始化
//3.打开文件
//3.1 获取文件流信息
//4.读取音频流
//5.找到对应的解码器
//6.设置音频信息，用来打开音频设备。
//7.打开音频设备。
//8.设置参数，供解码时候用
//9.打开解码器
//10.初始化队列
//11.SDL 播放音频
//12.循环读取音频帧，放到音频同步队列中
//13.回调函数中将从队列中取数据，解码后填充到播放缓冲区.
```

使用 ffmpeg+SDL 播放音频 demo

1. 定义所需的数据结构 -- 缓冲队列

packetqueue.h

```
#ifndef PACKETQUEUE_H
#define PACKETQUEUE_H


#ifdef __cplusplus
extern "C"{
#endif

#include <libavcodec/avcodec.h>
#include <libavformat/avformat.h>
#include <libswresample/swresample.h>
#include <SDL.h>

#ifdef __cplusplus
}
#endif

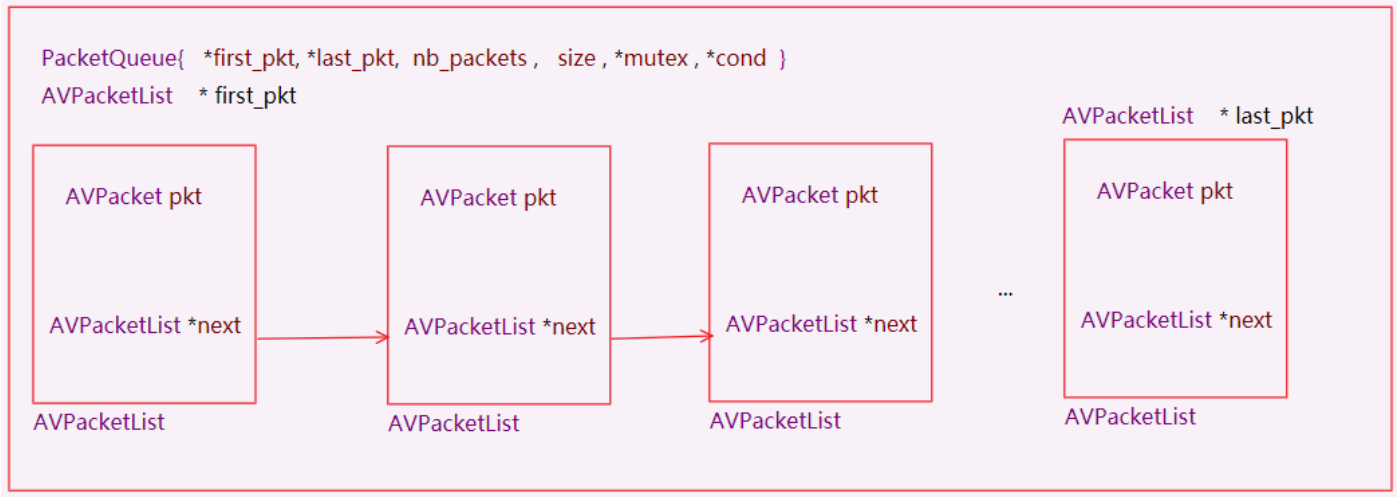
typedef struct PacketQueue
```

```
{
    AVPacketList    *first_pkt;     //队头的一个 packet, 注意类型不是 AVPacket
    AVPacketList    *last_pkt;      //队尾 packet
    int             nb_packets;     // paket 个数
    int             size;           //
    SDL_mutex       *mutex;         //
    SDL_cond        *cond;          // 条件变量

}PacketQueue;

void packet_queue_init(PacketQueue *queue);
int packet_queue_put(PacketQueue *queue, AVPacket *packet);
int packet_queue_get(PacketQueue *queue, AVPacket *pakcet, int block);

#endif // PACKETQUEUE_H
```



packetqueue.cpp

```
#include "packetqueue.h"

//初始化队列
void packet_queue_init(PacketQueue *queue)
{
    queue->first_pkt    = NULL;
    queue->last_pkt     = NULL;
    queue->nb_packets = 0;
    queue->size = 0;
    queue->mutex        = SDL_CreateMutex();
    queue->cond         = SDL_CreateCond();
}

//入队 -- 尾添加
int packet_queue_put(PacketQueue *queue, AVPacket *packet)
{
    AVPacketList   *pkt_list;
```

```
    // 包校验
    if (av_dup_packet(packet) < 0)
    {
        return -1;
    }

    pkt_list = (AVPacketList *)av_malloc(sizeof(AVPacketList));
    if (pkt_list == NULL)
    {
         return -1;
    }

    pkt_list->pkt  = *packet;
    pkt_list->next  = NULL;

    //上锁
    SDL_LockMutex(queue->mutex);

    if (queue->last_pkt == NULL)    //空队
    {
         queue->first_pkt = pkt_list;
    }
    else
    {
        queue->last_pkt->next = pkt_list;
    }

    queue->last_pkt = pkt_list;
    queue->nb_packets++;
    queue->size += packet->size;
    SDL_CondSignal(queue->cond); //添加完发送条件变量的信号--没有信号阻塞(生产者消费者)

    SDL_UnlockMutex(queue->mutex);

    return 0;
}

/// 出队--头删除
/// queue 传入队列指针 pkt 输出类型的参数, 返回结果.
/// block 表示是否阻塞 为 1 时 队列为空阻塞等待
int packet_queue_get(PacketQueue *queue, AVPacket *pkt, int block)
{
    AVPacketList   *pkt_list = NULL;
    int            ret = 0;
```

```
    SDL_LockMutex(queue->mutex);

    while(1)
    {
        pkt_list = queue->first_pkt;
        if (pkt_list != NULL)         //队不空，还有数据
        {
            queue->first_pkt = queue->first_pkt->next;    //pkt_list->next
            if (queue->first_pkt == NULL)
            {
                queue->last_pkt = NULL;
            }

            queue->nb_packets--;
            queue->size -= pkt_list->pkt.size;
            *pkt = pkt_list->pkt;          // 复制给 packet。
            av_free(pkt_list);
            ret = 1;
            break;
        }
        else if (block == 0)
        {
            ret = 0;
            break;
        }
        else
        {
            SDL_CondWait(queue->cond, queue->mutex);
        }
    }

    SDL_UnlockMutex(queue->mutex);
    return ret;
}
```

2.主函数解码并播放

main.cpp

```
#include <iostream>

using namespace std;

#include <stdio.h>
#include <stdlib.h>
```

```
#include <string.h>
#include <assert.h>
#include <unistd.h>

#ifdef __cplusplus
extern "C"
{
#endif

#include <libavcodec/avcodec.h>
#include <libavformat/avformat.h>
#include <libswresample/swresample.h>
#include <SDL.h>

#ifdef __cplusplus
}
#endif

#include "packetqueue.h"

#define AVCODEC_MAX_AUDIO_FRAME_SIZE    192000  //1 second of 48khz 32bit audio
#define SDL_AUDIO_BUFFER_SIZE   1024    //

#define FILE_NAME               "F:/Kugou/106.mp4"
#define ERR_STREAM              stderr
#define OUT_SAMPLE_RATE         44100

AVFrame         wanted_frame;
PacketQueue     audio_queue;
int             quit = 0;
//回调函数
void audio_callback(void *userdata, Uint8 *stream, int len);
//解码函数
int audio_decode_frame(AVCodecContext *pcodec_ctx, uint8_t *audio_buf, int buf_size);

//找 auto_stream
int find_stream_index(AVFormatContext *pformat_ctx, int *video_stream, int
*audio_stream);

#undef main
int main(int argc, char *argv[])
{
    //0.申请变量
    //AV 文件视频流的”文件指针”
    AVFormatContext     *pFormatCtx = NULL;
    int                 audioStream = -1;//解码器需要的流的索引
    AVCodecContext      *pCodecCtx = NULL;//解码器
```

```
AVCodec             *pCodec = NULL; //解码器
AVPacket            packet ;        // 解码前的数据
AVFrame             *pframe = NULL; //解码之后的数据
char                filename[256] = FILE_NAME;

//SDL
SDL_AudioSpec       wanted_spec;  //SDL 音频设置
SDL_AudioSpec       spec ; //SDL 音频设置

//1.ffmpeg 初始化
av_register_all();

//2.SDL 初始化
if (SDL_Init(SDL_INIT_VIDEO | SDL_INIT_AUDIO | SDL_INIT_TIMER))
{
    fprintf(ERR_STREAM, "Couldn't init SDL:%s\n", SDL_GetError());
    exit(-1);
}

//3.打开文件
if (avformat_open_input(&pFormatCtx, filename, NULL, NULL) != 0)
{
     fprintf(ERR_STREAM, "Couldn't open input file\n");
     exit(-1);
}

//3.1 获取文件流信息
if (avformat_find_stream_info(pFormatCtx, NULL) < 0)
{
     fprintf(ERR_STREAM, "Not Found Stream Info\n");
     exit(-1);
}

//显示文件信息，十分好用的一个函数
av_dump_format(pFormatCtx, 0, filename, false);

//4.读取音频流
if (find_stream_index(pFormatCtx, NULL, &audioStream) == -1)
{
    fprintf(ERR_STREAM, "Couldn't find stream index\n");
    exit(-1);
}
printf("audio_stream = %d\n", audioStream);

//5.找到对应的解码器
pCodecCtx = pFormatCtx->streams[audioStream]->codec;
pCodec = avcodec_find_decoder(pCodecCtx->codec_id);
```

```
if (!pCodec)
{
    fprintf(ERR_STREAM, "Couldn't find decoder\n");
    exit(-1);
}

//6.设置音频信息，用来打开音频设备。
wanted_spec.freq        = pCodecCtx->sample_rate;    //采样率
wanted_spec.format      = AUDIO_S16SYS; //音频采样格式 表示 16 位
wanted_spec.channels    = pCodecCtx->channels;       //通道数
wanted_spec.silence     = 0;    //设置静音值
wanted_spec.samples     = SDL_AUDIO_BUFFER_SIZE;   //预期的采样点数
wanted_spec.callback    = audio_callback;//回调函数
wanted_spec.userdata    = pCodecCtx;//回调函数参数

//wanted_spec:想要打开的
//spec:实际打开的,可以不用这个，函数中直接用 NULL,下面用到 spec 用 wanted_spec 代替。
//这里会开一个线程，调用 callback。
//SDL_OpenAudioDevice->open_audio_device(开线程)->SDL_RunAudio->fill(指向 callback 函数)
//7.打开音频设备。
//参数: 第一个为空表示默认设备 第二个参数 0 表示输出 1 表示输入
//参数: 第三个用于指定所需音频格式的 SDL_AudioSpec 结构体指针
//参数: 第四个用于接收实际打开的音频设备信息的 SDL_AudioSpec 结构体指针
//可以认为第三个参数是输入，第四个参数是根据设备和输入得到的输出
//参数: 第五个指示 SDL 可以在所需的规格中进行哪些更改，例如允许更改采样率或声道数等。
//如果不需要更改，则可以将此参数设置为 0。
SDL_AudioDeviceID id = SDL_OpenAudioDevice( NULL ,0,&wanted_spec, &spec,0);
if( id < 0  ) //第二次打开 audio 会返回-1
{
   fprintf(ERR_STREAM, "Couldn't open Audio: %s\n", SDL_GetError());
   exit(-1);
}

//8.设置参数，供解码时候用，swr_alloc_set_opts 的 in 部分参数
wanted_frame.format         = AV_SAMPLE_FMT_S16;
wanted_frame.sample_rate    = spec.freq;
//描述音频通道布局，用于指示某个音频流中各个声道的排列方式和数量
wanted_frame.channel_layout = av_get_default_channel_layout(spec.channels);
wanted_frame.channels       = spec.channels;

//9.打开解码器，初始化 AVCondecContext，以及进行一些处理工作。
avcodec_open2(pCodecCtx, pCodec, NULL);

//10.初始化队列
packet_queue_init(&audio_queue);

//11. SDL 播放声音 0 播放
```

```
    SDL_PauseAudioDevice(id,0);

    //12.循环读取音频帧(读一帧数据)放入音频同步队列
    while(av_read_frame(pFormatCtx, &packet) >= 0) //读一个 packet，数据放在 packet.data
    {
        if (packet.stream_index == audioStream)
        {
            packet_queue_put(&audio_queue, &packet);
        }
        else
        {
            av_free_packet(&packet);
        }
    }
    while( audio_queue.nb_packets != 0)
    {
        SDL_Delay(100);
    }
    //回收空间
    avcodec_close(pCodecCtx);
    avformat_close_input(&pFormatCtx);
    printf("play finished\n");
    return 0;
}


//注意 userdata 是前面的 AVCodecContext.
//len 表示一次发送多少。
//audio_buf_size：一直为样本缓冲区的大小，wanted_spec.samples.（一般每次解码这么多，文件不同，这个值不同）
//audio_buf_index：  标记发送到哪里了。
//这个函数的工作模式是:
//1. 解码数据放到 audio_buf，大小放 audio_buf_size。
//2. 调用一次 callback 只能发送 len 个字节,而每次取回的解码数据可能比 len 大，一次发不完。
//3. 发不完的时候，会 len 为 0，不继续循环，退出函数，继续调用 callback，进行下一次发送。
//4. 由于上次没发完，这次不取数据，发上次的剩余的，audio_buf_size 标记发送到哪里了。
//5. 注意，callback 每次一定要发且仅发 len 个数据，否则不会退出。
//如果没发够，缓冲区又没有了，就再取。发够了，就退出，留给下一个发，以此循环。
//三个变量设置为 static 就是为了保存上次数据，也可以用全局变量，但是感觉这样更好。

//13.回调函数中将从队列中取数据，解码后填充到播放缓冲区.
void audio_callback(void *userdata, Uint8 *stream, int len)
{
    AVCodecContext *pcodec_ctx    = (AVCodecContext *) userdata;

    int len1, audio_data_size;
```

```
    static uint8_t audio_buf[(AVCODEC_MAX_AUDIO_FRAME_SIZE * 3) / 2];
    static unsigned int audio_buf_size = 0;
    static unsigned int audio_buf_index = 0;

/*   len 是由 SDL 传入的 SDL 缓冲区的大小，如果这个缓冲未满，我们就一直往里填充数据 */

/*  audio_buf_index 和 audio_buf_size 标示我们自己用来放置解码出来的数据的缓冲区，*/
/*   这些数据待 copy 到 SDL 缓冲区， 当 audio_buf_index >= audio_buf_size 的时候意味着我*/
/*   们的缓冲为空，没有数据可供 copy，这时候需要调用 audio_decode_frame 来解码出更
/*   多的桢数据 */
    while (len > 0)
    {
        if (audio_buf_index >= audio_buf_size) {
            audio_data_size = audio_decode_frame(pcodec_ctx,
audio_buf,sizeof(audio_buf));
            /* audio_data_size < 0 标示没能解码出数据，我们默认播放静音 */
            if (audio_data_size < 0) {
                /* silence */
                audio_buf_size = 1024;
                /* 清零，静音 */
                memset(audio_buf, 0, audio_buf_size);
            } else {
                audio_buf_size = audio_data_size;
            }
            audio_buf_index = 0;
         }
        /*  查看 stream 可用空间，决定一次 copy 多少数据，剩下的下次继续 copy */
        len1 = audio_buf_size - audio_buf_index;
        if (len1 > len) {
            len1 = len;
        }

        memset( stream , 0 , len1);
        //混音函数 sdl 2.0 版本使用该函数 替换 SDL_MixAudio
        SDL_MixAudioFormat(stream, (uint8_t *) audio_buf + audio_buf_index,
                           AUDIO_S16SYS,len1,100);

        len -= len1;
        stream += len1;
        audio_buf_index += len1;
    }
}


//对于音频来说，一个 packet 里面，可能含有多帧(frame)数据。
int audio_decode_frame(AVCodecContext *pcodec_ctx, uint8_t *audio_buf, int buf_size)
```

```
{

    static AVPacket pkt;
    static uint8_t *audio_pkt_data = NULL;
    static int audio_pkt_size = 0;
    int len1, data_size;
    int sampleSize = 0;

    AVCodecContext *aCodecCtx = pcodec_ctx;
    AVFrame *audioFrame = NULL;
    PacketQueue *audioq = &audio_queue;

    static struct SwrContext   *swr_ctx = NULL;
    int convert_len;
    int n = 0;

    for(;;)
    {
        if( quit ) return -1;

        if(packet_queue_get(audioq, &pkt, 0) <= 0) //一定注意
        {
            return -1;
        }
        audioFrame = av_frame_alloc();
        audio_pkt_data = pkt.data;
        audio_pkt_size = pkt.size;
        while(audio_pkt_size > 0)
        {
            if( quit ) return -1;

            int got_picture;
            memset(audioFrame, 0, sizeof(AVFrame));

            int ret =avcodec_decode_audio4( aCodecCtx, audioFrame, &got_picture, &pkt);
            if( ret < 0 ) {
                printf("Error in decoding audio frame.\n");
                exit(0);
            }
//一帧一个声道读取数据是 nb_samples , channels 为声道数 2 表示 16 位 2 个字节
            data_size = audioFrame->nb_samples * wanted_frame.channels * 2;

            if( got_picture )
            {
                if (swr_ctx != NULL)
                {
                     swr_free(&swr_ctx);
```

```
                    swr_ctx = NULL;
                }

                swr_ctx = swr_alloc_set_opts(NULL, wanted_frame.channel_layout,
                        (AVSampleFormat)wanted_frame.format,wanted_frame.sample_rate,
                        audioFrame->channel_layout,(AVSampleFormat)audioFrame->format,
                        audioFrame->sample_rate, 0, NULL);

                //初始化
                if (swr_ctx == NULL || swr_init(swr_ctx) < 0)
                {
                    printf("swr_init error\n");
                    break;
                }
                convert_len = swr_convert(swr_ctx, &audio_buf,
                                AVCODEC_MAX_AUDIO_FRAME_SIZE,
                                (const uint8_t **)audioFrame->data,
                                audioFrame->nb_samples);

            }

            audio_pkt_size -= ret;

            if (audioFrame->nb_samples <= 0)
            {
                continue;
            }

            av_free_packet(&pkt);
            return data_size  ;
        }
        av_free_packet(&pkt);
    }
}

int find_stream_index(AVFormatContext *pformat_ctx, int *video_stream, int
*audio_stream)
{
    assert(video_stream != NULL || audio_stream != NULL);

     int i = 0;
     int audio_index = -1;
     int video_index = -1;

     for (i = 0; i < pformat_ctx->nb_streams; i++)
     {
          if (pformat_ctx->streams[i]->codec->codec_type == AVMEDIA_TYPE_VIDEO)
```

```
        {
            video_index = i;
        }
        if (pformat_ctx->streams[i]->codec->codec_type == AVMEDIA_TYPE_AUDIO)
        {
            audio_index = i;
        }
    }

    //注意以下两个判断有可能返回-1.
    if (video_stream == NULL)
    {
        *audio_stream = audio_index;
        return *audio_stream;
    }

    if (audio_stream == NULL)
    {
        *video_stream = video_index;
         return *video_stream;
    }

    *video_stream = video_index;
    *audio_stream = audio_index;

    return 0;
}
```